Rubrica: Systems Design

Learning a black box, one proven rule at a time

Reto 2 · La caja negra · rubrica.dev · Kevin Vicent · v19 · For the judges and any engineer who rebuilds the replica

The black box is an unknown decision procedure. We find its rules with experiments. We prove each rule against the real service. Then we compile the rules into a deterministic replica. Most agent systems need an LLM judge. Ours has an exact judge for every behavior it probes: the original service. What it never probes is the open risk, so we measure it.

1. Problem and goals

An undocumented service receives text messages on `POST /msg` (body `{"m": "…"}`) and keeps state between requests. It also has `/__reset` and `/health`. Each token has its own state. The service changes during the day, without notice.

Goals: ① the same status and body as the original on the 100 hidden tests (40 single requests, 60 sequences of 2–12 steps), run on the **20:00 version** · ② a spec and an experiment log in which every claim cites evidence, re-checked by code before we submit (a false claim subtracts points) · ③ stay correct after the service changes. Every observable behavior is part of the contract (Hyrum's Law), errors included.

Non-goals: fixing the original (we copy its errors) · state that survives a restart (every test starts with a reset) · latency under the 5 s step limit · headers (the tests do not compare them).

Design principle: the README contract is the architecture. Everything the service returns is **data**: learned, versioned and replaceable. No part of the design depends on a specific operation, field or error code.

Assumption	How we test it
Same state and same request give the same response	Replay a sequence twice
Messages have learnable structure (operations, fields, checks)	Fit rules; confirm with probes
When several errors apply, one fixed precedence decides	Combine errors in pairs
Generated values (IDs, counters) follow a reproducible rule	Compare sequences across resets

Hard cases: computed values (for example integrity checks) · error precedence · generated IDs · repeated operations · limits · auth variants · unknown routes · **changes during the day**.

Priority: 40 of the 100 tests are single requests right after a reset, so behaviors reachable from the initial state are probed first.

2. Interface: what the replica must do

- `GET /health` → 200 when ready · `POST /__reset` → initial state · **any other route or method** → exactly the original's status and body.
- **Auth:** `Authorization: Bearer <team token>` on every call; another token, or none, gets the original's answer for that case.
- **No matching rule** → a default chosen from evidence, marked *supuesto* in the spec. One process; < 5 s per step; never calls the original: that scores 0 (README), so a test fails on any outbound connection.

3. Architecture: dual-process, one ontology

We use Kahneman's terms. **System 2** is slow and deliberate: the offline loop that discovers the rules. **System 1** is fast and automatic: the deployed replica that applies them.

1. **Agent harness (System 2, offline).** A *harness* is the code around a model that gives it tools, memory and limits. A person and a reasoning model use only recorded tools (`probe`, `sequence`, `fuzz`, `diff`, `shrink`: minimize a failing sequence); each request is logged with its hypothesis. **No request without a prediction:** a code gate blocks requests with no prediction, already answered by the log, or over budget. Rate-limit responses come from the infrastructure, so they are dropped, never learned. When several candidate rules fit the evidence, the harness sends the request that best separates them; an unknown rule gets at most 50 probes, then becomes a documented default (*supuesto*). A learned formula is trusted only after it predicts inputs it has never seen.
2. **Service ontology (memory).**
   - **What it is:** an explicit model of a domain: its concepts and the relations between them. Ours models the black box.
   - **What it holds:** operations, fields, state, rules, errors and evidence. A rule applies to an operation, needs preconditions, returns a response, changes state, outranks other rules and is proven by log lines. Its confidence uses the README's levels: *confirmado* (proven, repeated), *probable* (seen once or inferred), *supuesto* (untested default).
   - **What we use it for:** run the replica, generate `SPEC.md`, build tests from evidence, choose the next probes from its gaps, and **compare versions: most changes are new data; a new operation is one more rule.**
3. **Replica (System 1, runtime).** HTTP adapter → serial queue (one request at a time, so state changes in a fixed order, as in the original) → decision core. Compiling sorts the rules by learned precedence and indexes them by operation; **the first matching rule decides.** State is in memory, one state per token.

Decision	Chosen	Rejected, and why
Runtime	**Compiled rule table:** System 2 knowledge made automatic; no probabilities	*LLM or classifier:* a formal protocol has nothing to perceive; not exact or repeatable. *Record and replay:* fails on new inputs.
Discovery	**A person and a model in a harness**, exact judge	*Free LLM chat:* no proof or log. *Autonomous agent:* false claims lose points. *Manual curl:* too slow.
Knowledge	**Typed ontology**, one source for code, spec and tests	*Separate spec, code, tests:* diverge after a change. *RAG over logs:* approximate. *OWL/RDF:* heavy for one day; typed records hold the same relations.
Checks	**Exact checks in code, cheapest first:** cache → corpus → live original; drift triage too	*LLM judge:* false passes; the original is a free, exact judge.
Server	**Minimal server we control; one always-on instance**	*Framework defaults:* different 404/405 bodies. *Autoscaling:* split state.

4. How we measure: evals first

The **recorded corpus** guides fixes; **held-out tests** only measure, so the score stays honest (Goodhart's Law). A different generator builds them (mutations of the README examples, structure-agnostic fuzz), so they do not share our blind spots.

- **Differential test:** replays every recorded sequence against the replica and compares status and body with the original's. Target: 100% on the current version's corpus; **≥ 95% on held-out tests**.
- **Drift canary:** every 10 min, 10 requests to the original (5 fixed on the most-tested paths, 5 rotating through the corpus). A difference from the recorded answer, except values that change on every run (such as timestamps), means the service changed. Detect in ≤ 10 min on fixed paths, one rotation elsewhere. **Fix in ≤ 45 min:** re-probe the changed rules, new version, rerun the differential test, redeploy.
- **Coverage:** operation × precondition × error matrix, low-confidence rules first. **Discovery rate:** new behaviors per 100 probes; an area is done at zero. **Budget:** 8,000 requests/day, 10/s (README); half is held back to re-learn after changes, since the 20:00 version is the one scored.
- **Designed tests:** each error alone and **in pairs**, boundary values ±1, case and whitespace variants, field count ±1, repeated operations, limits.
- **Validations:** 3 hidden-set runs, spent only if the exact-match rate did not decrease (~15:15, ~17:15, ~19:30).

Risks: a rule outside our candidates (add candidates) · budget runs out (cache, never repeat) · late drift (canary until 20:00).